

PhishGuard: An Explainable Machine Learning Framework for Real-Time Phishing URL Detection Using Feature Engineering and Ensemble Learning

1. Introduction

Brief Overview of the Project Idea

PhishGuard is being developed as a web-based security platform for detecting potentially malicious and phishing URLs. Phishing attacks are a major cybersecurity threat in which attackers create deceptive websites or URLs to trick users into revealing sensitive information such as login credentials, financial details, and personal data. The system analyzes a submitted URL using multiple URL-based features and a machine learning model to classify it as legitimate or phishing. Feature engineering is used to extract relevant characteristics from the URL, and ensemble learning is used to improve classification reliability, while an explainable prediction mechanism accompanies each result so that users can understand why a URL was classified as suspicious or legitimate. The system consists of a React-based frontend for submitting URLs and viewing results, and a Django backend that processes the URL, extracts features, and communicates with the prediction component; scan history is maintained for later review.

Need / Motivation Behind Selecting the Topic

The growing use of online services, digital payments, and social media has increased users' exposure to malicious links, which are often designed to closely resemble legitimate websites. Manually checking URLs or relying only on static blacklists is insufficient for identifying newly generated or previously unseen phishing URLs. This project explores machine learning, feature engineering, and explainable prediction as a more adaptive and transparent approach to automated phishing detection.

Relevance of the Project in the Current Scenario

A machine-learning-based detection system can analyze URL characteristics and identify suspicious patterns more efficiently than manual inspection. The project is relevant because it combines web development, cybersecurity, machine learning, and explainable artificial intelligence into a single practical application, while also serving as an educational framework for applying machine learning to real-world cybersecurity problems.

2. Problem Statement

Problem: Existing phishing detection approaches often struggle to identify newly generated, previously unseen, or deliberately obfuscated malicious URLs in real time, and a prediction without an understandable explanation is difficult for users to trust or interpret.

Existing Issues in Phishing URL Detection Systems

- **Zero-day / unseen URLs:** newly created malicious URLs are absent from blacklists or reputation databases.
- **Evolving attack patterns:** attackers continuously modify URL structures and use obfuscation to evade static rules.
- **High false positives / negatives:** distinguishing legitimate from malicious URLs accurately remains a major challenge, especially with a limited feature set.
- **Lack of explainability:** most automated systems provide only a bare prediction or warning without justifying it.
- **Manual analysis is not scalable:** manual inspection is time-consuming and dependent on human expertise.

3. Literature Review / Related Work

Existing phishing-detection literature broadly follows three directions. Blacklist and whitelist-based approaches are fast and produce few false positives, but are reactive — a URL is blocked only after it has already been reported and verified. Heuristic and rule-based approaches use hand-crafted rules on URL structure, such as the feature assessment by Mohammad, Thabtah, and McCluskey (2012), and are reasonably interpretable, but are brittle against new obfuscation patterns and require constant manual updating. Machine-learning-based approaches trained on URL and host features generalize better to unseen URLs — Chiew et al. (2019) proposed a hybrid ensemble feature-selection framework, and Patil et al. (2025) evaluated stacking and voting-based boosting ensembles for robust malicious-URL classification. Explainability techniques such as SHAP (Lundberg & Lee, 2017) have since been combined with comparative classifiers, as in Ismail, Nasution, and Gea (2026).

However, published work has rarely combined ensemble classifiers with an explainability layer inside one deployable, real-time system — the gap that PhishGuard addresses.

4. Objectives of the Project

- Design and develop a web-based system for detecting phishing and legitimate URLs.
- Collect, preprocess, and prepare a suitable dataset for phishing URL classification.
- Extract meaningful URL-based lexical, host-based, and structural features using feature engineering techniques.
- Develop, train, and evaluate machine learning classification models, including ensemble techniques, for reliable phishing detection.
- Integrate an explainable prediction mechanism (SHAP) that highlights the characteristics contributing to each classification.
- Integrate the trained model with a backend API for real-time URL prediction.
- Maintain scan history and provide an organized interface for reviewing previous URL analyses.

5. Scope of the Project

In Scope

- Extraction of URL-based lexical, host-based, and structural features without paid third-party APIs.
- Training and evaluation of multiple supervised classifiers and an ensemble model, with an explainable prediction for every result.
- A REST API and web-based frontend for real-time URL checking and scan-history review.

Out of Scope

- Browser-extension development and full webpage/DOM content analysis (planned as future work).
- Mobile-native application development and integration with paid threat-intelligence services.

6. Software and Hardware Requirements

➤ Programming Language(s)

- Python
- JavaScript
- HTML
- CSS

➤ Tools / Frameworks

- React.js (Vite) — for the frontend interface.
- Django — for backend development and API integration.
- Django REST Framework — for REST API development.
- Scikit-learn — for machine learning model development and evaluation.
- Pandas — for dataset processing and analysis.
- NumPy — for numerical processing.
- Jupyter Notebook / Google Colab — for experimentation and model development.
- Visual Studio Code — for development.
- Git and GitHub — for version control and project management.

➤ Machine Learning

- Random Forest, XGBoost, Support Vector Machine (SVM), LightGBM, CatBoost, Logistic Regression — base classification models.
- Voting Classifier — for ensemble learning across the base models.
- SHAP (SHapley Additive exPlanations) — for generating explainable predictions.
- Scikit-learn metrics — Accuracy, Precision, Recall, F1-score, ROC-AUC, and Confusion Matrix for model evaluation.

> Database

MongoDB will be used as the primary database for storing application data, including URL records, extracted features, predictions, model performance metrics, and scan history. MongoEngine will be used as the object-document mapper to integrate MongoDB with the Django backend.

> Operating System

Windows 10/11 or Linux.

> Hardware

A PC/Laptop with at least 8 GB RAM is recommended for development, dataset processing, model experimentation, and testing. A stable internet connection will be useful for development, testing, and deployment-related activities.

7. The Proposed System

The proposed PhishGuard system consists of five interconnected modules: URL Input and Validation, Feature Extraction and Engineering, Machine Learning Classification and Ensemble Learning, Explainable Prediction, and Result Presentation and Scan History.

- **I. URL Input & Validation** – users submit a URL through the web interface, and the backend validates the input before further processing.
- **II. Feature Extraction & Engineering** – lexical, domain-based, and structural features (URL length, special characters, IP usage, subdomains, HTTPS, etc.) are extracted and converted into a model-ready representation.
- **III. ML Classification & Ensemble Learning** – trained classifiers are combined through ensemble learning to classify the URL as Phishing or Legitimate.
- **IV. Explainable Prediction** – SHAP identifies which features most influenced each prediction, keeping the output transparent rather than a bare label.
- **V. Result Presentation & Scan History** – results are returned via the API and displayed on the frontend, and scan records are stored for later review.

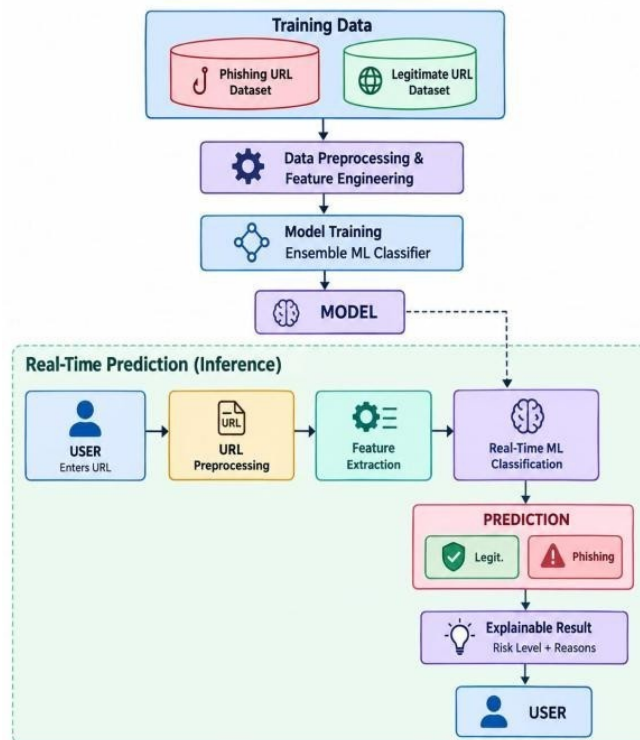


Figure 1: Block Diagram of the Proposed PhishGuard Phishing URL Detection System

Simulation

- Simulate URL submission, validation, and feature extraction across legitimate and phishing URL patterns.
- Simulate machine learning predictions and ensemble-based classification using prepared test data.
- Simulate explainable-prediction output and scan-history creation and retrieval through the backend API.

Analysis

- Analyze the effectiveness of selected features and compare individual classifiers against the ensemble.
- Evaluate model performance using accuracy, precision, recall, F1-score, and confusion-matrix analysis.
- Assess the usefulness of the explainable-prediction information for end users.

8. Current Implementation Status and Goals for the Final Phase

The Phase 1 baseline of PhishGuard — URL input and validation, deterministic lexical and hostname feature extraction, a transparent heuristic verdict, the Django REST API, MongoDB-backed scan history, and the React/JSX frontend — has been implemented and verified. Guided by the project supervisor, the following milestones define the remaining work for the final phase of the project:

- **a. Feature Fine-Tuning & Dataset Expansion** – clean up any lingering data noise from the Kaggle datasets, ensure robust handling of missing values, and lock in the final feature-vector representation ahead of full-scale model training.
- **b. Model Optimization & Baseline Freeze** – train and optimize the ensemble model (Random Forest + XGBoost + SVM) to lock in target performance metrics — aiming for consistent maintenance of close to 95% accuracy and a 0.971 ROC-AUC — and integrate the SHAP explainability module so every classification produces stable, interpretable output. These figures are targets to be validated once training and evaluation are complete; the current baseline uses a heuristic predictor and does not yet report a trained-model accuracy.
- **c. Full-Stack Integration** – ensure seamless, low-latency communication between the React frontend, the Django REST API, and MongoDB so that URL scanning, risk assessment, and history logging operate reliably in real time.
- **d. Proactive Prototyping (Browser Extension / API Gateway)** – begin developing a browser-extension or lightweight plugin wrapper so users can check URLs directly while browsing, fulfilling a key part of the project's future scope.
- **e. Testing & Security Audits** – carry out structured unit, API, integration, and performance testing, along with security checks for input sanitization and protection against malformed-URL or injection-based failures.
- **f. Deployment & Cloud Hosting** – containerize the application using Docker and deploy the full stack to a cloud platform (e.g., AWS) paired with a production MongoDB instance.

9. Expected Outcomes

Project Deliverables

- A functional, full-stack web-based prototype for real-time phishing URL detection.
- A trained and evaluated ensemble machine learning model with explainable (SHAP) predictions.
- A REST API and responsive frontend for real-time and historical URL analysis.
- A prototype browser-extension or API-gateway component, and a containerized, cloud-deployable version of the application.
- Complete project documentation, testing results, and performance analysis.

Benefits / Contributions

- Provides an automated, explainable approach to identifying potentially phishing URLs, giving users clear, evidence-based reasons behind each verdict rather than an opaque label.
- Demonstrates the practical integration of feature engineering, ensemble learning, and explainable AI in cybersecurity.
- Establishes a maintainable full-stack foundation (React + Django + MongoDB) that can be extended with minimal rework, serving as a base for future work such as a browser extension.

10. Conclusion

PhishGuard demonstrates how a verified, network-free heuristic baseline can evolve into an explainable, ensemble-based phishing detection system. The Phase 1 implementation establishes a solid full-stack foundation — URL validation, feature extraction, a Django REST API, MongoDB-backed history, and a React frontend — and the goals defined for the final phase, namely dataset expansion, model optimization and explainability, full-stack integration, a browser-extension prototype, security testing, and cloud deployment, provide a clear, supervisor-guided path to project completion.

By explaining why a URL is flagged rather than returning a black-box label, PhishGuard is positioned to build user trust and support faster security decision-making, growing from a transparent, rule-based prototype into a fully explainable, ensemble-driven phishing detection system suitable for real-world demonstration and further research.

11. Timeline of the Project

The project will be completed through the following tentative phases, spanning the full development cycle from literature review to final documentation:

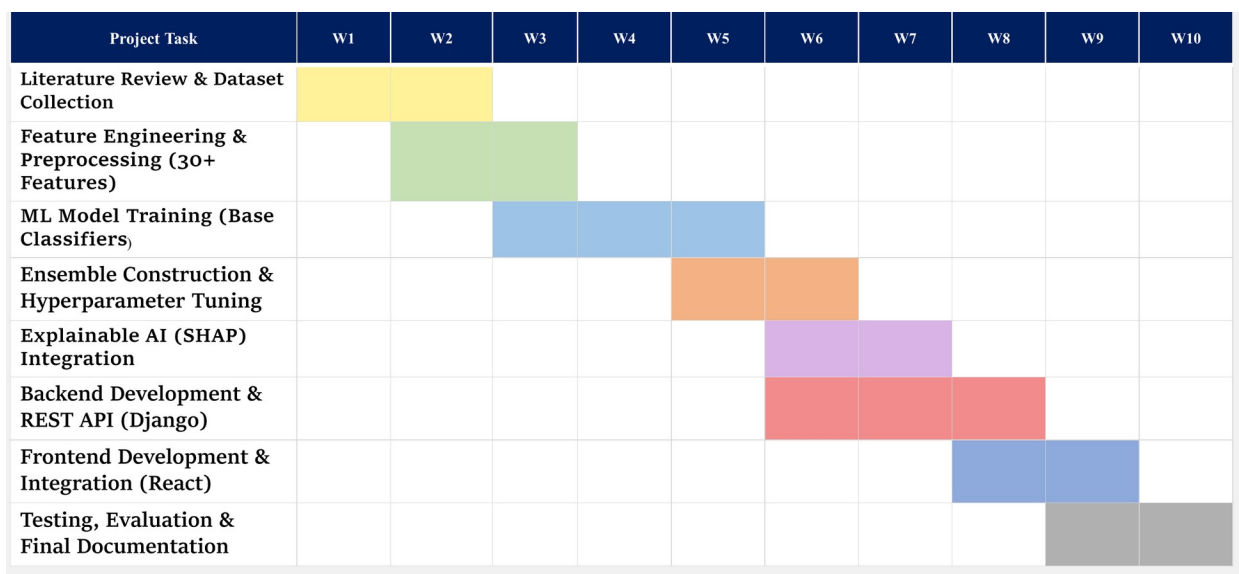


Figure 2: Gantt Chart Representation of the Project Phases and Timeline (Tentative)